Reflection of Templates

Document #:	P3240R0 [Latest] [Status]
Date:	2024-10-16
Project:	Programming Language C++
Audience:	EWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com> Andrei Alexandrescu, NVIDIA <andrei@nvidia.com> Daveed Vandevoorde, EDG <daveed@edg.com> Michael Garland, NVIDIA <mgarland@nvidia.com>

Contents

§ 1 Motivation

A key characteristic that makes a reflection facility powerful is *completeness*, i.e., the ability to reflect the entirety of the source language. Current proposals facilitate reflection of certain declarations in a namespace or a `struct`/`class`/`union` definition. Although [P2996R7]'s `members_of` metafunction includes template members (function template and class template declarations), it does not offer primitives for reflection of template declarations themselves. In this proposal, we aim to define a comprehensive API for reflection of C++ templates.

A powerful archetypal motivator—argued in [P3157R1] and at length in the CppCon 2024 talk "Reflection Is Not Contemplation"— is the *identity* metafunction. This function, given a class type, creates a replica of it, crucially providing the ability to make minute changes to the copy, such as changing its name, adding or removing members, changing the signature of existing member functions, and so on. By means of example:

```
class Widget {
    int a;
public:
    template <class T> requires (std::is_convertible_v<T, int>)
    Widget(const T&);
    const Widget& f(const Widget&) const &;
    template <class T>
    void g();
};

consteval {
    identity(^^Widget, "Gadget");
}
// Same effect as this handwritten code:
// class Gadget {
//     int a;
// public:
//     template <class T> requires (std::is_convertible_v<T, int>)
//     Gadget(const T&);
//     const Gadget& f(const Gadget&) const &;
//     template <class T>
//     void g();
// };
```

While an exact copy of a type is not particularly interesting—and although a compiler could easily provide a primitive to do so, such a feature would miss the point—the ability to deconstruct a type into its components (bases, data members, member functions, nested types, nested template declarations, etc.)

and reassemble them in a different context enables a vast array of applications. These include various forms of instrumentation, creating parallel hierarchies as required by several design patterns, generating arbitrary subsets of an interface (the *powerset* of an interface), logging, tracing, debugging, timing measurements, and many more. We consider `identity`'s ability to perform the roundtrip from a type to its components and back to the type the quintessential goal of reflection, which is at the same time the proof of reflection's completeness and the fountainhead of many of its applications.

In order to reflect on nontrivial C++ code and splice it back with controlled modifications, it is necessary to reflect all templates—class templates and specializations thereof, function templates, variable templates, and alias templates. This is not an easy task; templates pose unique challenges to a reflection engine because, by their nature, they are only partially analyzed semantically; for example, neither parameter types nor the return type of a function template can be represented as reflections of types because some of those types are often not known until the template is instantiated. The same thinking goes for a variety of declarations that can be found inside a class template; C++ templates are more akin to patterns by which code is to be generated than to standalone, semantically verified code. For that reason, the reflection facilities proposed in P2996 intended for non-templated declarations are not readily applicable to templates.

Conversely, reflection code would be seriously hamstrung if it lacked the ability to reflect on template code (and subsequently manipulate and generate related code), especially because templates are ubiquitous in today's C++ codebases. Furthermore, limiting reflection to instantiations of class templates (which would fit the charter of P2996) does not suffice because class templates commonly define inner function templates (e.g., every instantiation of `std::vector` defines several constructor templates and other function templates such as `insert` and `emplace_back`). Therefore, to the extent reflection of C++ declarations is deemed useful, reflection of templates is essential.

Given that many elements of a template cannot be semantically analyzed early (at template definition time), this proposal adopts a consistent strategy for reflecting components of template declarations: each element of a template declaration—such as the parameter list, constraints, and template argument list—is represented as a *token sequence*, as proposed in [P3294R2]. We consider token sequences a key enabler of the ability to introspect templates as proposed in this paper. Though operating with token sequences is a departure from P2996's *modus operandi*, we paid special attention to ensure seamless interoperation with it.

As lightly structured representations of components of template declarations, token sequences have several important advantages:

— *Simplicity:* C++ template declarations and the rules for substitution and looking up symbols are quite complex. Adding new language elements dedicated to handling, for example, reflection of dependent types would complicate the language immensely. In contrast, token sequences are a simple, uniform, and effective means to represent any part of a template declaration because they fundamentally are created from, and consist of, C++ source code.

— *Interoperation:* Token sequences will seamlessly interoperate with, and take advantage of, all primitives and facilities introduced in P3294 related to token sequences.

— *Code generation ability:* A key aspect of reflection is the ability to "hook into" code by generating new code that copies existing functionality and adds elements such as code instrumentation, logging,

or tracing. Generating instrumented templates from components of existing template declarations is simple and straightforward—as simple as any use of token sequences for code generation.

— *Implementation friendliness:* Token sequences do not require heavy infrastructure or a specific implementation design. In addition, this proposal grants important freedom and latitude to compilers, as detailed in the next section.

§ 1.1 The "As-If" Rule for Token Sequences Returned by Reflection Metafunctions

Most metafunctions proposed in this paper return token sequences. Some implementations may not preserve the exact tokens as initially present in source code for a variety of reasons. First, it is possible that an implementation "normalizes" code that may have different forms in source, for example replacing `template<class T>` with the equivalent `template<typename T>` or vice versa. Second, a compiler may run some simple front-end processing during tokenization, replacing e.g. `+1` or `01` with `1`. Third, compilers may look up some types early and replace them with the internal representation thereof, which is fully constant-folded and with all default arguments substituted—again, making it tenuous to return looked-up types exactly in the form originally present in source code. Fourth, some implementations are known to parse templates eagerly and drop the tokens early in the processing pipeline. For such a compiler, producing the tokens would be difficult and inefficient.

Given these realities of current compiler implementations, demanding that the compiler returns the exact tokens present in the initial source code may entail unnecessary burden on some implementations. We therefore stipulate that metafunctions that return token sequences are bound by the following minimal requirements:

— Metafunctions that return token sequences originating from existing code must return code that, when spliced back into source, compiles and runs with the same semantics as the original source code.

— Implementations are allowed to return token sequences that contain nonstandard code (including handles into compiler-internal data structure), again as long as the result of splicing has the same semantics as the original source code.

— Returning empty token sequences is always observed.

— There is no other guaranteed assumption about metafunctions that return tokens.

In particular, user code should not assume token sequences returned by implementation-defined metafunctions compare equal with tokens assumed to be equivalent. The only guarantee is that splicing the tokens back has the same effect. We hope that this proposal sets a useful precedent for all future standard metafunctions that return tokenized representation of existing code.

Producing a printable token stream on demand remains important for debugging purposes. We consider it a quality of implementation matter.

§ 1.2 Example: Logging and Forwarding

As a conceptual stylized example, consider the matter of creating a function template `logged::func` for any free function template `func` in such a way that `logged::func` has the same signature and semantics as `func`, with the added behavior that it logs the call and its parameters prior to execution. We illustrate the matter for function templates because it is more difficult (and more complete) than the case of simple functions.

```
template <typename T>
requires std::is_copy_constructible_v<T>
void fun(const T& value) { ... }

consteval void make_logged(std::meta::info f) {
    queue_injection(^^{
        namespace logged {
            \tokens(copy_signature(f, name_of(f))) {
                  logger("Calling " + name_of(f) + " with arguments: ",
        \tokens(params_of(f)));
                return \tokens(make_fwd_call(f));
            }
        }  // end manespace logged
    });
}

consteval {
    make_logged(^^fun);
}

// Equivalent hand-written code:
// namespace logged {
//     template <typename T>
//     requires std::is_copy_constructible_v<T>
//     void fun(const T& value) {
//         logger("Call to ", "fun", " with arguments: ", value);
//         return fun<T>(std::forward<T>(value));
//     }
// }
```

We identify a few high-level metafunctions that facilitate such manipulation of template declarations. `copy_signature(info f, std::string_view name)` produces the token sequence corresponding to the signature of the function template reflected by `f`, under a new name. (The example above uses the same name in a distinct namespace.) The template parameters, `requires` clause, function parameters, and return type are all part of the copied tokens. (In the general case, we do want to have separate access to each element for customization purposes, so `copy_signature` would combine more fine-grained primitives.) Next, `params_of(f)` produces a comma-separated list of the parameters of `f`, with pack expansion suffix where applicable. Finally, `make_fwd_call` creates the tokens of a call to the function reflected by `f` with the appropriate insertion of calls to `std::forward`, again with pack expansion suffix where appropriate.

In order to be able to define metafunctions such as `copy_signature`, `params_of`, and `make_fwd_call`, we need access to the lower-level components of a template declaration.

§ 2 Metafunctions for Template Declarations

All template declarations (whether a class template or specialization thereof, a function template, an alias template, or a variable template), have some common elements: a template parameter list, an optional list of attributes, and an optional template-level `requires` clause.

If our goal were simply to splice a template declaration back in its entirety—possibly with a different name and/or with normalized parameter names—the introspection metafunction `copy_signature(^^X, "Y")` showcased above could be defined as an implementation-defined primitive that simply splices the entire declaration of template `X` to declare a new template `Y` that is (save for the name) identical to `X`. However, most often code generation needs to tweak elements of the declaration—for example, adding a conjunction to the `requires` clause or eliminating the `deprecated` attribute. Simply returning an opaque token sequence of the entire declaration and leaving it to user-level code to painstakingly parse it once again would be inefficient and burdensome; therefore, we aim to identify structural components of the declaration and define primitives that return each in turn. That way, we offer unbounded customization opportunities by allowing users to mix and match elements of the original declaration with custom code. Once we have all components of the template declaration, implementing `copy_signature` as a shorthand for assembling them together is trivially easy.

In addition to template parameters, `requires` clause, and attributes, certain template declarations have additional elements as follows:

- Class templates and variable templates may add explicit specializations and partial specializations
- Function templates may create overload sets (possibly alongside regular functions of the same name)
- Alias templates contain a declarator
- Member variables of class templates, and also variable templates, have a type and an optional initializer
- Function template declarations may contain these additional elements:
  - `inline` specifier
  - `static` linkage
  - `noexcept` clause (possibly predicated)
  - cvref qualifiers
  - `explicit` specifier (possibly predicated)
  - `constexpr` or `consteval` specifier
  - return type
  - function parameters
  - trailing `requires` clause
  - function template body

Armed with a these insights and with the strategy of using token sequences throughout, we propose the following reflection primitives for templates.

§ 2.1 Synopsis

The declarations below summarize the metafunctions proposed, with full explanations for each following. Some (where noted) have identical declarations with metafunctions proposed in P2996, to which we propose extended semantics.

```
namespace std::meta {
    //  Multiple explicit specializations and/or partial specializations
    consteval auto template_alternatives_of(info) -> vector<info>;
    // Template parameter normalization prefix
    constexpr string_view template_parameter_prefix;
    // Template parameter list
    consteval auto template_parameters_of(info) -> vector<info>;
    // Attributes - extension of semantics in P3385
    consteval auto attributes_of(info) -> vector<info>;
    // Template-level requires clause
    consteval auto requires_clause_of(info) -> info;
    // Is this the reflection of the primary template declaration?
    consteval auto is_primary_template(info) -> bool;
    // Arguments of an explicit specialization or partial specialization
    consteval auto specialization_arguments_of(info) -> vector<info>;
    // Bases of a class template
    consteval auto template_bases_of(info) -> vector<info>;
      // Type of a data member in a class template or of a variable
         template
    consteval auto template_data_type(info);
    // Type of a data member in a class template
    consteval auto template_data_initializer(info) -> info;
    // Declarator part of an alias template
    consteval auto alias_template_declarator(info) -> info;
    // Inline specifier present?
    consteval auto is_inline(info) -> bool;
    // cvref qualifiers and others - extensions of semantics in P2996
    consteval auto is_const(info) -> bool;
    consteval auto is_explicit(info) -> bool;
    consteval auto is_volatile(info) -> bool;
    consteval auto is_rvalue_reference_qualified(info) -> bool;
    consteval auto is_lvalue_reference_qualified(info) -> bool;
    consteval auto is_static_member(info) -> bool;
    consteval auto has_static_linkage(info) -> bool;
    consteval auto is_noexcept(info) -> bool;
    // predicate for `noexcept`
    consteval auto noexcept_of(info) -> info;
    // `explicit` present? - extension of P2996
    // predicate for `explicit`
    consteval auto explicit_specifier_of(info) -> info;
    // `constexpr` present?
    consteval auto is_declared_constexpr(info) -> bool;
    // `consteval` present?
    consteval auto is_declared_consteval(info) -> bool;
```

```
    // Function template return type
    consteval auto template_return_type_of(info) -> info;
    // Function template parameter normalization prefix
    constexpr string_view function_parameter_prefix;
    // Function parameters
           consteval   auto   template_function_parameters_of(info)   ->
        vector<info>;
    // Trailing `requires`
    consteval auto trailing_requires_clause_of(info) -> info;
    // Function template body
    consteval auto body_of(info) -> info;
}
```

§ 2.1.1 `template_alternatives_of`

```
vector<info> template_alternatives_of(info);
```

In addition to general template declarations, a class template or a variable template may declare explicit specializations and/or partial specializations. Example:

```
// Primary template declaration
template <typename T, template<typename> typename A, auto x> class C;
// Explicit specialization
template <> class C<int, std::allocator, 42> {};
// Partial specialization
template <typename T> class C<T, std::allocator, 100> {};
```

Each specialization must be accessible for reflection in separation from the others; in particular, there should be a mechanism to iterate `^^C` to reveal `info` handles for the three available variants (the primary declaration, the explicit specialization, and the partial specialization). Each specialization may contain the same syntactic elements as the primary template declaration. In addition to those, explicit specializations and partial specializations also contain the specialization's template arguments (in the code above: `<int, std::allocator, 42>` and `<T, std::allocator, 100>`, respectively), which we also set out to be accessible through reflection.

Given the reflection of a class template or variable template, `template_alternatives_of` returns the reflections of all explicit specializations and all partial specializations thereof (including the primary declaration). Declarations are returned in syntactic order. This is important because a specialization may depend on a previous one, as shown below:

```
// Primary declaration
template <class T> struct A { ... };
// Explicit specialization
template <> struct A<char> { ... };
// Explicit specialization, depends on the previous one
template <> struct A<int> : A<char> { ... };
```

```
// Partial specialization, may depend on both previous ones
template <class T, class U> struct A<std::tuple<T, U>> : A<T>, A<U> {
        ... };
```

As a consequence of the source-level ordering, the primary declaration's reflection is always the first element of the returned vector. Example:

```
template <typename T> class C;
template <> class C<int> {};
template <class T> class C<std::pair<int, T>> {};

// Get reflections for the primary and the two specializations
constexpr auto r = template_alternatives_of(^^C);
static_assert(r.size() == 3);
static_assert(r[0] == ^^C);
```

§ 2.1.2 `template_parameters_of`, `template_parameter_prefix`

```
vector<info> template_parameters_of(info);
constexpr string_view template_parameter_prefix = unspecified;
```

Given the reflection of a template, `template_parameters_of` returns an array of token sequences containing the template's parameters (a parameter pack counts as one parameter). Template parameter names are normalized by naming them the concatenation of `std::meta::template_parameter_prefix` and an integral counter starting at 0 and incremented with each parameter introduction, left to right. For example, `template_parameters_of(^^A)` yields the token sequences `^^{typename _T0}`, `^^{_T0 _T1}`, and `^^{auto... _T2}`. Note that the second parameter has been described as `_T0 _T1` (renaming the dependent name correctly). The template parameter prefix `std::meta::template_parameter_prefix` is an implementation-reserved name such as `"_T"` or `"__t"`. (Our examples use `"_T"`.)

For example:

```
template <typename T, T x, auto... f> class A;

static_assert(template_parameters_of(^^A).size() == 3);
// r0 is ^^{typename _T0}
constexpr auto r0 = template_parameters_of(^^A)[0];
// r1 is ^^{_T0 x}
constexpr auto r1 = template_parameters_of(^^A)[1];
// r2 is ^^{auto... _T0}
constexpr auto r2 = template_parameters_of(^^A)[2];
```

As noted above, an implementation may return token sequences equivalent to those in the source code, e.g. `^^{class _T0}` instead of `^^{typename _T0}`.

Defaulted parameters, if any, are included in the token sequences returned. For example:

```
template <typename U = int> class A;

// r is ^^{typename _T0 = int}
constexpr auto r = template_parameters_of(^^A)[0];
```

For explicit specializations of class templates, `template_parameters_of` returns an empty vector:

```
template <typename T> class A;
template <> class A<int>;

// r refers to A<int>
constexpr auto r = template_alternatives_of(^^A)[1];   // definition
        below
static_assert(template_parameters_of(r).empty());
```

If a non-type template parameter has a non-dependent type, the tokens returned for that parameter include the fully looked-up name. For example:

```
namespace Lib {
    struct S {};
    template <S s> class C { ... };
}
struct S {};  // illustrates a potential ambiguity

// r is ^^{::Lib::S _T0}, not ^^{S _T0}
constexpr auto r = template_parameters_of(^^Lib::C)[1];
```

Given a class template `A` that has explicit instantiations and/or partial specializations declared, the call `template_parameters_of(^^A)` is not ambiguous—it refers to the primary declaration of the template.

Calling `template_parameters_of` against a non-template entity, as in `template_parameters_of(^^int)` or `template_parameters_of(^^std::vector<int>)`, fails to evaluate to a constant expression.

§ 2.1.3 `attributes_of`

```
vector<info> attributes_of(info);
```

Returns all attributes associated with `info`. The intent is to extend the functionality of `attributes_of` as defined in P3385 to template declarations. We defer details and particulars to that proposal.

§ 2.1.4 `requires_clause_of`

```
info requires_clause_of(info);
```

Given the reflection of a template, returns the template-level `requires` clause as a token sequence. The names of template parameters found in the `requires` clause are normalized. Non-dependent identifiers are returned in fully-qualified form. For example:

```
struct X { ... };
template <typename T>
requires (sizeof(X) > 256 && std::is_default_constructible_v<T>)
class A;

//        Returns        `^^{(sizeof(::X)        >        1        &&
        ::std::is_default_constructible_v<_T0>)}`
constexpr auto r1 = requires_clause_of(^^A);

template <typename T>
requires (sizeof(T) > 1 && std::is_default_constructible_v<T>)
using V = std::vector<T>;

//        Returns        `^^{(sizeof(_T0)        >        1        &&
        ::std::is_default_constructible_v<_T0>)}`
constexpr auto r2 = requires_clause_of(^^V);

template <class T>
requires (std::is_convertible_v<int, T>)
auto zero = T(0);

// Returns `^^{(::std::is_convertible_v<int, _T0>)}`
constexpr auto r3 = requires_clause_of(^^zero);
```

Calling `requires_clause_of` against a template that has no such clause returns an empty token sequence. Calling `requires_clause_of` against a non-template, as in `requires_clause_of(^^int)` or `requires_clause_of(^^std::vector<int>)`, fails to evaluate to a constant expression.

§ 2.1.5 `is_primary_template`

```
bool is_primary_template(info);
```

Returns `true` if `info` refers to the primary declaration of a class template or variable template, `false` in all other cases (including cases in which the query does not apply, such as `is_primary_template(^^int)`). In particular, this metafunction is useful for distinguishing between a primary template and its explicit specializations or partial specializations. For example:

```
template <class> class A;
template <> class A<int>;
static_assert(is_primary_template(^^A));
static_assert(is_primary_template(template_alternatives_of(^^A)[0]));
static_assert(!is_primary_template(template_alternatives_of(^^A)[1]));
static_assert(!is_primary_template(^^int));
```

§ 2.1.6 `specialization_arguments_of`

```
vector<info> specialization_arguments_of(info);
```

For an explicit specialization or a partial specialization of a class template or variable template, returns that specialization's arguments. For example:

```
template <typename T> class A;
template <> class A<int>;
template <typename T> class A<std::tuple<int, T>>;
// Get the reflection of the first explicit specialization
constexpr auto r1 = template_alternatives_of(^^A)[1];
// Get the argument of that specialization, i.e. ^^{int}
constexpr auto r11 = specialization_arguments_of(r1)[0];
// Get the reflection of the second explicit specialization
constexpr auto r2 = template_alternatives_of(^^A)[2];
// Get the arguments of that specialization, i.e. ^^{std::tuple<int, T>}
constexpr auto r21 = specialization_arguments_of(r2)[0];
```

If an explicit specialization uses default arguments, those will be made part of the return of `specialization_arguments_of`, even if they are not syntactically present in the source code:

```
template <typename T = int> class A;
// Explicitly specialize A<int>.
template <> class A<>;
// Get the explicit specialization
constexpr auto r = template_alternatives_of(^^A)[1];
// r1 will be ^^{int}
constexpr auto r1 = specialization_arguments_of(r)[0];
```

Note that `specialization_arguments_of` may return an empty vector. For example:

```
template <typename... Ts> class A;
template <> class A<>;
constexpr auto r = template_alternatives_of(^^A)[1];
// No specialization arguments present
static_assert(specialization_arguments_of(r).empty());
```

The metafunction `specialization_arguments_of` has a similar role but is qualitatively different from `template_arguments_of` in P2996 because the latter returns resolved entities (types, values, or template names), whereas the former returns token sequences of the arguments.

Non-dependent types returned are always returned in fully namespace-qualified form. Example:

```
namespace N {
    struct S {};
    template <typename T> class A;
    // Explicitly specialize A<int>.
    template <> class A<S>;
}
struct S {};  // to illustrate a possible ambiguity

// Get the explicit specialization
constexpr auto r = template_alternatives_of(^^N::A)[1];
// r1 will be ^^{::N::S}, not ^^{S}
constexpr auto r1 = specialization_arguments_of(r)[0];
```

If `specialization_arguments_of` is called with an `info` that is not an explicit template specialization, the call fails to resolve to a constant expression.

§ 2.1.7 `template_bases_of`

```
vector<info> template_bases_of(info);
```

Returns the bases of a class template, in declaration order, as token sequences. Access specifiers are part of the returned tokens in such a way that reassembling the result of `template_bases_of` as a comma-separated list produces a valid base classes specifier. Example:

```
class B1 { ... };
template <typename T> class B2 { ... };
template <typename T> class C : public B1, public B2<T> { ... };

// Contains ^^{public ::B1}
constexpr auto r1 = template_bases_of(^^C)[0];
// Contains ^^{public ::B2<_T0>}
constexpr auto r2 = template_bases_of(^^C)[1];
```

§ 2.1.8 `template_data_type`

```
info template_data_type(info);
```

Given the reflection of a data member of a class template or of a variable template, returns the type of the member as a token sequence. Returning a type reflection, as is the case for P2996's `type_of`, is not possible in most cases. Example:

```
template <typename T>
struct A {
    const T& x;
};
```

```
template <class T>
auto zero = T(0);

// Call P2996's nonstatic_data_members_of extended for templates
// r1 contains ^^{_T0 const&}
constexpr auto r1 = template_data_type(nonstatic_data_members_of(^^C)
        [0]);
// r2 contains ^^{_T0(0)}
constexpr auto r2 = template_data_type(^^zero);
```

§ 2.1.9 `template_data_initializer`

```
info template_data_initializer(info);
```

Returns the initializer part of a data member of a class template or of a variable template declaration, as a token sequence. Example:

```
template <class T>
struct S {
    T x = T("hello");
    T y;
}

template <class T>
auto zero = T(0);

// r1 contains `^^{_T0("hello")}`
constexpr auto r1 = template_data_initializer(nonstatic_data_member-
        s_of(^^S)[0]);
// r2 contains `^^{}`
constexpr auto r2 = template_data_initializer(nonstatic_data_member-
        s_of(^^S)[1]);
// r3 contains `^^{_T0(0)}`
constexpr auto r3 = template_data_initializer(^^zero);
```

If `template_data_initializer` is called with an `info` that is not an alias template, the call fails to resolve to a constant expression.

§ 2.1.10 `alias_template_declarator`

```
info alias_template_declarator(info);
```

Returns the declarator part of an alias template declaration, as a token sequence. Example:

```
template <typename T> using V = std::vector<T>;
```

```
// Returns `^^{::std::vector<_T0, ::std::allocator<_T0>>}`
constexpr auto r = alias_template_declarator(^^V);
```

If `alias_template_declarator` is called with an `info` that is not an alias template, the call fails to resolve to a constant expression.

§ 2.1.11 `overloads_of`

```
vector<info> overloads_of(info);
```

Returns an array of all overloads of a given function, usually passed in as a reflection of an identifier (e.g., `^^func`). All overloads are included, template and nontemplate. To distinguish functions from function templates, `is_function_template` (P2996) can be used. Example:

```
void f();
template <typename T> void f(const T&);

// Name f has two overloads present.
static_assert(overloads_of(^^f).size() == 2);
```

The order of overloads in the results is the same as encountered in source code.

If a function `f` has only one declaration (no overloads), `overloads_of(^^f)` returns a vector with one element, which is equal to `^^f`. Therefore, for such functions, `^^f` and `overloads_of(^^f)[0]` can be used interchangeably.

Each `info` returned by `overloads_of` is the reflection of a fully resolved symbol that doesn't participate in any further overload resolution. In other words, for any function `f` (overloaded or not), `overloads_of(overloads_of(^^f)[i])` is the same as `overloads_of(^^f)[i]` for all appropriate values of `i`.

§ 2.1.12 `is_inline`

```
bool is_inline(info);
```

Returns `true` if and only if the given reflection handle refers to a function, function template, or namespace that has the `inline` specifier. In all other cases, such as `is_inline(^^int)`, returns false.

Note that this metafunctions has applicability beyond templates as it applies to regular functions and namespaces. Because of this it is possible to migrate this metafunction to a future revision of P2996.

§ 2.1.13 `is_const`, `is_explicit`, `is_volatile`, `is_rvalue_reference_qualified`, `is_lvalue_reference_qualified`, `is_static_member`, `has_static_linkage`, `is_noexcept`

```
bool is_const(info);
bool is_explicit(info);
bool is_volatile(info);
bool is_rvalue_reference_qualified(info);
bool is_lvalue_reference_qualified(info);
bool is_static_member(info);
bool has_static_linkage(info)
bool is_noexcept(info);
```

Extends the homonym metafunctions defined by P2996 to function templates.

In order to avoid confusion, in case a predicate is present for `is_noexcept` or `is_explicit`, the call fails to resolve to a constant expression. Therefore, for templates it is best to use the `noexcept_of` and `explicit_specifier_of` metafunctions (below), which support all cases.

§ 2.1.14 `noexcept_of`

```
info noexcept_of(info);
```

Returns the `noexcept` clause of a function or function template, if any, as a token sequence, as follows:

— if no `noexcept` clause is present, returns the empty token sequence;

— if `noexcept` is unconditional, returns the token sequence `^^{noexcept}`;

— if `noexcept` is conditional (e.g., `noexcept(expression)`), returns the conditional noexcept in tokenized form;

— if `noexcept` would not be applicable (e.g., `noexcept(^^int)`), the call fails to evaluate to a constant expression.

§ 2.1.15 `explicit_specifier_of`

```
info explicit_specifier_of(info);
```

Returns the `explicit` specifier of the given `info` of a constructor. If the constructor has no explicit specifier, returns the empty sequence. If the constructor has an unconditional `explicit` specifier, returns the token sequence `^^{explicit}`. If the constructor has a predicated `explicit(constant-expression)`, returns the token sequence `^^{explicit(constant-expression)}`. All nondependent identifier in the token sequence are fully namespace-qualified.

§ 2.1.16 `is_declared_constexpr`, `is_declared_consteval`

```
bool is_declared_constexpr(info);
bool is_declared_consteval(info);
```

Returns `true` if and only if `info` refers to a declaration that is `constexpr` or `consteval`, respectively. In all other cases, returns `false`.

§ 2.1.17 `template_return_type_of`

```
info template_return_type_of(info);
```

Returns the return type of the function template represented by `info`. This function is similar with `return_type_of` in P3096. However, one key difference is that `return_type_of` returns the reflection of a type, whereas `template_return_type_of` returns the token sequence of the return type (given that the type is often not known before instantiation or may be `auto&` etc).

§ 2.1.18 `template_function_parameters_of`, `function_parameter_prefix`

```
vector<info> template_function_parameters_of(info);
constexpr string_view function_parameter_prefix;
```

Returns an array of parameters (type and name for each) of the function template represented by `info`, as token sequences. Parameter names are normalized by naming them the concatenation of `std::meta::function_parameter_prefix` and an integral counter starting at 0 and incremented with each parameter introduction, left to right. Normalizing parameter names makes it easy for splicing code to generate forwarding argument lists such as the metafunctions `params_of` and `make_fwd_call` in the opening example.

This function bears similarities with `parameters_of` in P3096. However, one key difference is that `parameters_of` returns reflection of types, whereas `template_return_type_of` returns the token sequence of the return type (given that the type is often not known before instantiation or may be `auto&` etc).

§ 2.1.19 `trailing_requires_clause_of`

```
info trailing_requires_clause_of(info);
```

Given the `info` of a function template, returns the trailing `requires` clause of the declaration as a token sequence. Example:

```
template <typename T>
requires (sizeof(T) > 8)
void f() requires (sizeof(T) < 64);

// Will contain ^^{ (sizeof(T) < 64) }
constexpr auto r = trailing_requires_clause_of(^^f);
```

§ 2.1.20 `body_of`

```
info body_of(info);
```

Given the `info` of a function template, returns the function's body as a token sequence, without the top-level braces (which can be added with ease if needed). All parameter names are normalized and all non-dependent identifiers are fully qualified. Example:

```
template <typename T>
void f(T x) {
    return x < 0 ? -x : x;
}

// Will contain ^^{ return _p0 < 0 ? -_p0 : _p0; }
constexpr auto r = body_of(^^f);
```

§ 3 Metafunctions for Iterating Members of Class Templates

The metafunctions described above need to be complemented with metafunctions that enumerate the contents of a class template. Fortunately, P2996 already defines number of metafunctions for doing so: `members_of`, `static_data_members_of`, `nonstatic_data_members_of`, `nonstatic_data_members_of`, `enumerators_of`, `subobjects_of`. To these, P2996 adds access-aware metafunctions `accessible_members_of`, `accessible_bases_of`, `accessible_static_data_members_of`, `accessible_nonstatic_data_members_of`, and `accessible_subobjects_of`. Here, we propose that these functions are extended in semantics to also support `info` representing class templates. The reflections of members of class templates, however, are not the same as the reflection of corresponding members of regular classes;

One important addendum to P2996 that is crucial for code generation is that when applied to members, these discovery metafunctions return members in lexical order. Otherwise, attempts to implement `identity` will fail for class templates. Consider:

```
template <typename T>
struct Container {
    using size_type = size_t;
    size_type size() const;
}
```

The reflections of template components returned by `members_of` and others can be inspected with template-specific primitives, not the primitives defined in P2996. For example, the data members returned by `nonstatic_data_members_of` can be inspected with `template_data_type` and `template_data_initializer`, but not with `offset_of`, `size_of`, `alignment_of`, or `bit_size_of` because at the template definition time the layout has not been created.

§ 4 Future Work

This proposal is designed to interoperate with [P2996R7], [P3157R1], [P3294R2], and [P3385R0]. Changes in these proposals may influence this proposal. Also, certain metafunctions proposed herein (such as `is_inline`) may migrate to these proposals.

The normalization scheme for template parameters and function parameters is rigid. We plan to refine it in future revisions of this document.

§ 5 References

[P2996R7] Wyatt Childers, Peter Dimov, Dan Katz, Barry Revzin, Andrew Sutton, Faisal Vali, and Daveed Vandevoorde. 2024-10-12. Reflection for C++26.

https://wg21.link/p2996r7

[P3157R1] Andrei Alexandrescu, Barry Revzin, Bryce Lelbach, Michael Garland. 2024-05-22. Generative Extensions for Reflection.

https://wg21.link/p3157r1

[P3294R2] Andrei Alexandrescu, Barry Revzin, and Daveed Vandevoorde. 2024-10-15. Code Injection with Token Sequences.

https://wg21.link/p3294r2

[P3385R0] Aurelien Cassagnes, Aurelien Cassagnes, Roman Khoroshikh, Anders Johansson. 2024-09-16. Attributes reflection.

https://wg21.link/p3385r0